

Unit 1

Introduction to Computer Architecture

- *Introduction to Computer Architecture and Organization*
- *Basic structure of Computers: Von Neumann Architecture*
- *Functional units of a computer system*
- *Types of computer architectures: SISD, SIMD, MIMD, MISD*
- *Performance metrics: Throughput, Latency, MIPS, FLOPS*
- *Machine language and assembly language*
- *Data representation: Number systems (Binary, Hexadecimal, Octal)*
- *Signed number representations (1's complement, 2's complement)*
- *Floating-point arithmetic and IEEE standards*
- *Overview of System Bus and Bus Interconnection*

Introduction to Computer Architecture and Organization:

Computer Organization and Architecture is used to design computer systems. Computer Architecture is considered to be those attributes of a system that are visible to the user like addressing techniques, instruction sets, and bits used for data, and have a direct impact on the logic execution of a program, it defines the system in an abstract manner.

Computer Architecture

- *Refers to the design principles and fundamental operational structure of a computer system.*
- *It focuses on the logical aspects of the hardware, like instruction sets, addressing modes, and system organization.*
- *It deals with the **structure, behavior, and design** of a computer system, focusing on how the CPU performs and how it interacts with memory and I/O devices.*

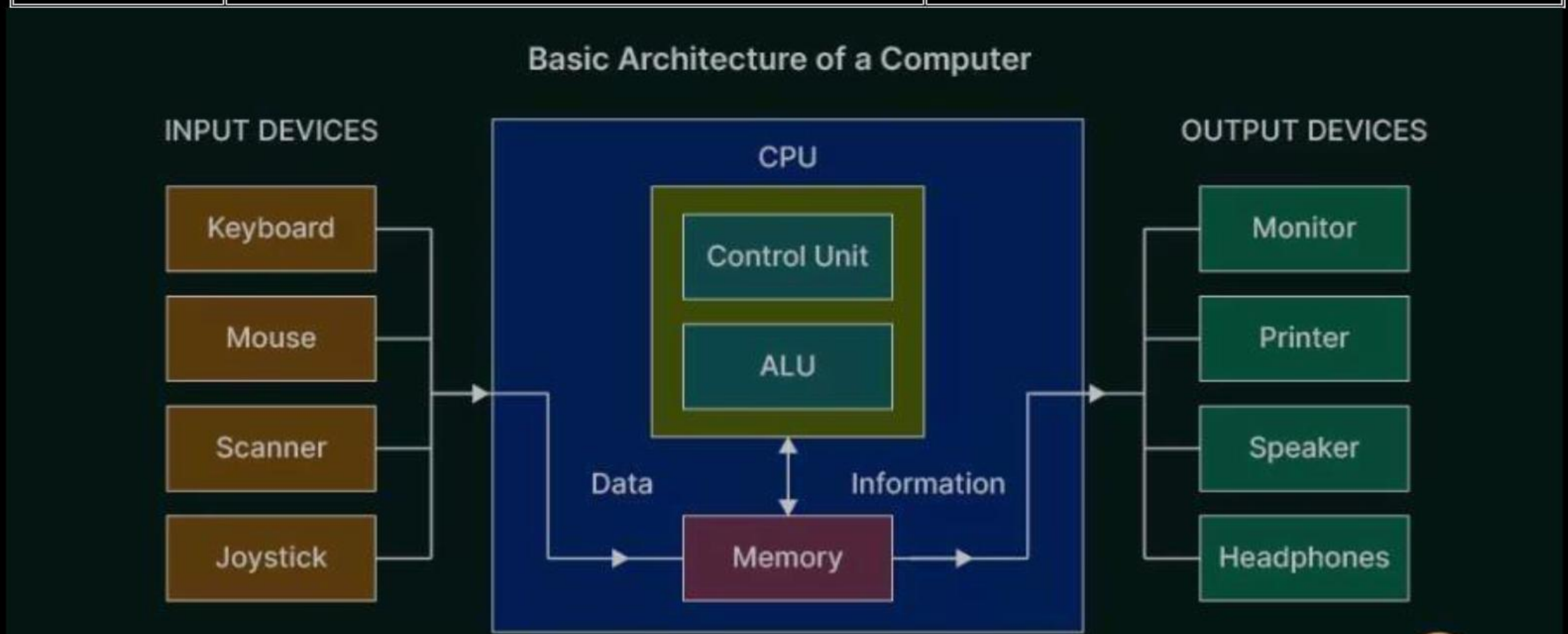
Computer Organization

- *Refers to the operational aspects of a computer, detailing how the hardware components are interconnected and work together.*
- *It includes details like the hardware components, data pathways, and control signals.*

Difference Between Computer Architecture and Computer Organization

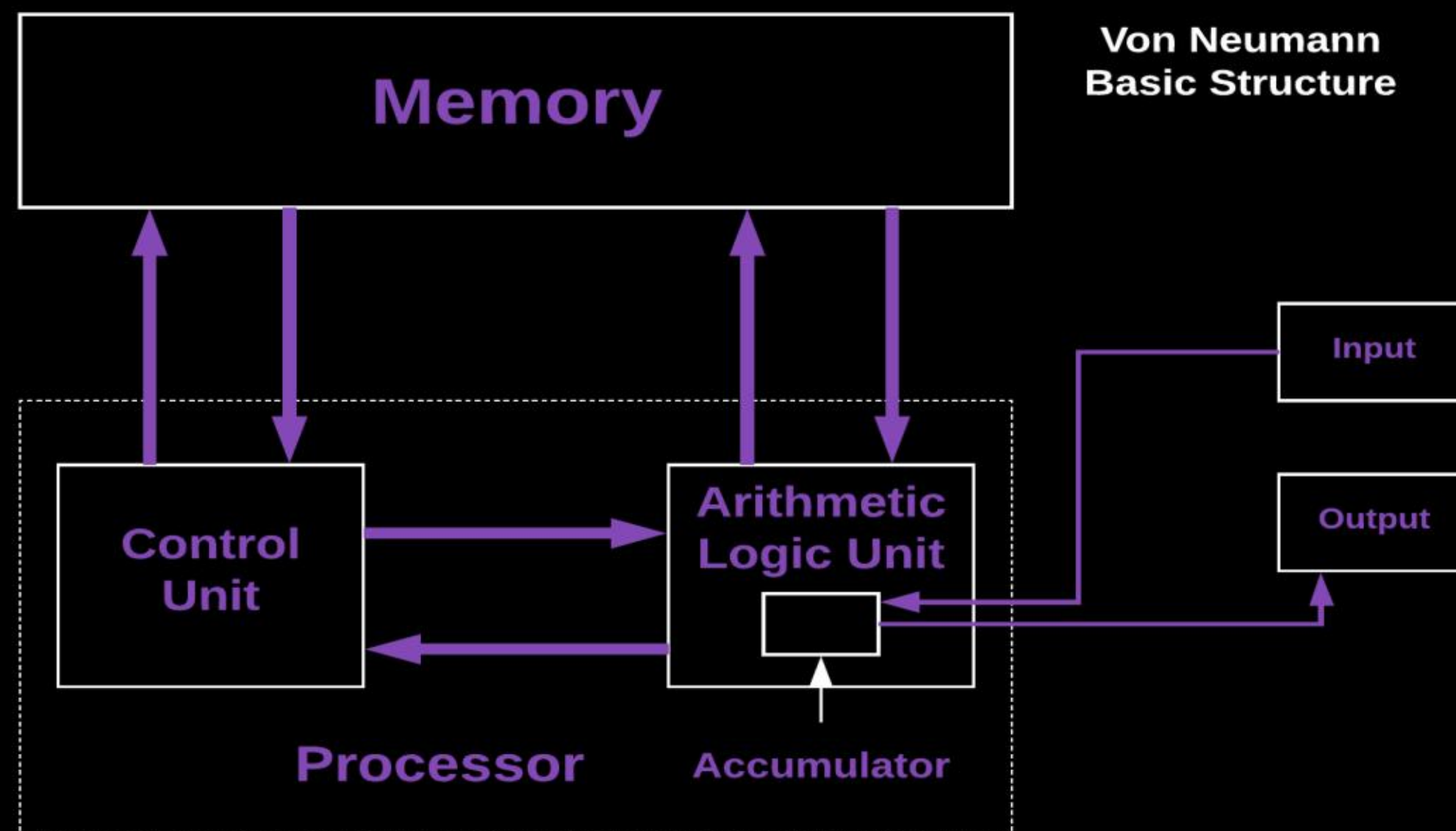
Criteria	Computer Architecture	Computer Organization
----------	-----------------------	-----------------------

Definition	Refers to the set of rules and design principles that describe how a computer works. It deals with the logical structure, behavior, and functionality of a computer system.	Refers to the operational components of a computer and how they are interconnected and work together to implement the architecture.
Focus	Focuses on the instruction set , data types, addressing modes, and system-level design.	Focuses on the implementation details , including data paths, control signals, and physical components.
Concerned With	High-level design aspects, like the Instruction Set Architecture (ISA) and system performance optimization.	Details of the physical hardware and the lower-level implementation, like circuits, ALU design , and microarchitecture.
Design Aspects	Involves designing the instruction set , processor capabilities, and memory hierarchy.	Involves designing the control unit , bus systems, I/O handling, and internal data transfer mechanisms.
User Perspective	Visible to the software developers as it defines the programmer's interface (instructions, registers, memory).	Mainly visible to hardware designers and engineers , focusing on efficient data handling and hardware utilization.
Example Components	Instruction set, data formats, and control mechanisms (e.g., pipelining, parallelism).	Register organization, control signals, cache design, data paths, and timing signals.
Design Level	Deals with abstract and logical design. Higher-level design decisions.	Deals with concrete and physical implementation. Lower-level technical details.
Examples	RISC vs CISC architecture, Harvard vs Von Neumann architecture.	Microarchitecture of Intel, AMD processors, ALU design, and control unit organization.



2. Basic Structure of Computers: Von Neumann Architecture

- The **Von Neumann Architecture**, proposed by John von Neumann, forms the basis of most modern computers.
- It is characterized by a **single memory** space used for both data and instructions, with a sequential execution of instructions.



The Von Neumann architecture is a computer architecture design, first described by John von Neumann in 1945

Key Components

- **CPU (Central Processing Unit):**

The brain of the computer, containing the Control Unit (CU) and Arithmetic Logic Unit (ALU) to decode and execute instructions and perform calculations.

- **Memory:**

A single memory unit that stores both the program's instructions and the data it uses.

- **I/O Devices:**

Input/Output interfaces that allow the computer to communicate with the user and other storage devices.

- **Buses:**

Communication pathways that carry data between the CPU, memory, and I/O devices.

Types of CPU Buses

- **Address Bus:**

This is a one-way pathway from the CPU to memory or I/O devices. It carries memory addresses, telling other components where data is located or where to send data.

- **Data Bus:**

This is a two-way pathway that carries the actual data between the CPU and other components. It handles numerical values for calculation, calculation results, and input/output signals.

- **Control Bus:**

This bus carries signals that manage and synchronize the operations of the system. It tells the address and data buses what to do, for example, whether to perform a read operation (from memory) or a write operation (to memory).

How it Works

1. ***Instruction Fetch:*** The CPU fetches an instruction from memory.
2. ***Instruction Decode:*** The Control Unit decodes the instruction.
3. ***Data Fetch:*** If the instruction requires data, the CPU fetches it from the same memory.
4. ***Execution:*** The ALU performs the required operation.
5. ***Data Storage:*** The result is stored back into memory.
6. ***Sequential Cycle:*** The cycle repeats, with the CPU fetching and executing instructions one after another from the shared memory.

Performance metrics: Throughput, Latency, MIPS, FLOPS

1. Throughput

Definition

- *Throughput is the amount of work or data processed by a system per unit of time.*
- *It measures the rate at which a system completes tasks or outputs results.*

Characteristics of Throughput

- Often measured in *tasks per second*, *transactions per second (TPS)*, *bytes per second (Bps)*, or *instructions per second*.
- High throughput indicates a system that can handle a larger volume of tasks or data quickly.
- Related to **system capacity**, but it does not account for how long an individual task takes.

Example

- A web server that handles **1,000 requests per second** has a throughput of **1,000 TPS**.
- A network with a throughput of **500 Mbps** can transfer 500 megabits of data per second.

Use Case

- Throughput is used to measure the performance of databases, web servers, networks, and data processing systems.

2. Latency

Definition

- Latency is the **time delay** between the initiation of a task and its completion, often referred to as **response time**.
- It measures the time taken for a system to **respond** to an input or request.

Characteristics of Latency

- Measured in units of **time**, such as milliseconds (ms), microseconds (μ s), or seconds (s).
- Lower latency means faster response time and better performance.
- Can be affected by factors like **network delays**, **processing time**, and **I/O operations**.

Example

- The time taken for a **ping** request to receive a response from a server is a measure of latency.
- In a database system, the time from when a query is sent to when the results are received is the latency.

Use Case

- Latency is crucial in **real-time systems**, gaming, video streaming, and financial trading, where quick response times are critical.
-

3. MIPS (Million Instructions Per Second)

Definition

- MIPS is a metric that measures the **speed of a processor** in executing a fixed number of **machine-level instructions** per second.
- It indicates the **instruction execution rate** of a CPU.

Characteristics of MIPS

- Measured in **millions of instructions per second**.
- Gives an estimate of the **raw speed** of a processor but does not consider the **complexity of instructions**.
- Not always a reliable metric for comparing different architectures (e.g., RISC vs. CISC), as the instruction sets may differ.

Example

- A CPU with a speed of **3 MIPS** can execute **3 million instructions per second**.
- A microcontroller that executes **2.5 MIPS** is capable of processing 2.5 million instructions every second.

Use Case

- MIPS is often used in benchmarking processors, especially in older computing systems, microcontrollers, and embedded systems.

Limitations

- **Instruction Complexity:** Different instructions take varying amounts of time, so MIPS does not always reflect actual performance.
- **Architecture Dependency:** MIPS comparisons are not valid across different CPU architectures (e.g., RISC vs. CISC).

4. FLOPS (Floating-Point Operations Per Second)

Definition

- FLOPS is a metric used to measure the **performance of a processor** in executing **floating-point arithmetic operations**.
- It evaluates the computational power of systems, especially for tasks involving **scientific calculations** and **numerical computations**.

Characteristics of FLOPS

- Measured in units like **GFLOPS** (GigaFLOPS, or billion FLOPS), **TFLOPS** (TeraFLOPS, or trillion FLOPS), and **PFLOPS** (PetaFLOPS, or quadrillion FLOPS).
- Primarily used to assess the performance of **supercomputers**, **GPUs**, and systems handling **complex mathematical calculations**.
- Indicates how well a system can handle **floating-point** calculations, which are common in scientific and AI applications.

Example

- *A processor with a performance of **1 TFLOPS** can perform **1 trillion floating-point operations per second**.*
- *An NVIDIA GPU with **15 TFLOPS** can execute 15 trillion floating-point calculations every second, making it suitable for AI and deep learning tasks.*

Use Case

- *FLOPS is used to evaluate the performance of **supercomputers**, scientific simulations, machine learning, and graphics processing.*

Limitations

- ***Limited Scope:** Only measures floating-point operations, not integer or logical operations.*
- ***Hardware Dependency:** Performance can vary based on the floating-point unit (FPU) and parallel processing capabilities of the hardware.*

Comparison Table: Throughput, Latency, MIPS, and FLOPS

Aspect	Throughput	Latency	MIPS	FLOPS
Definition	Rate of work or data processed per unit time.	Time delay between task initiation and completion.	Speed of executing machine instructions per second.	Speed of executing floating-point operations per second.
Measurement Unit	Tasks/sec, Bytes/sec, Requests/sec.	Time (ms, μ s, s).	Millions of Instructions Per Second (MIPS).	Floating-Point Operations Per Second (FLOPS).
Focus	System capacity and volume of processing.	Response time and quickness of task completion.	Instruction execution rate.	Floating-point computation performance.
Use Case	Networks, databases, web servers.	Real-time systems, gaming, video streaming.	Embedded systems, general-purpose CPUs.	Supercomputers, GPUs, scientific computing.
Limitation	Does not measure individual task time.	Does not reflect overall system capacity.	Not reliable for different architectures.	Only measures floating-point performance.

Examples and Use Cases

- Throughput in a Database System:**
 - A database server can handle **5,000 transactions per second (TPS)**. This high throughput indicates the system's capacity to handle large volumes of transactions efficiently.
- Latency in a Gaming Application:**
 - In online gaming, a **latency of less than 20 ms** is ideal for a smooth gaming experience. Higher latency results in noticeable lag and delayed responses.
- MIPS in a Microcontroller:**
 - A microcontroller running at **50 MIPS** can execute 50 million simple instructions per second, suitable for embedded applications like sensor data processing.
- FLOPS in Scientific Computing:**
 - A supercomputer performing at **1 PFLOPS** (1 petaflop) can handle **quadrillions of floating-point calculations per second**, making it ideal for complex simulations like weather forecasting or molecular modeling.

SISD, SIMD, MIMD, MISD Architectures

		Instruction Streams	
		one	many
Data Streams	one	SISD traditional von Neumann single CPU computer	MISD May be pipelined Computers
	many	SIMD Vector processors fine grained data Parallel computers	MIMD Multi computers Multiprocessors

1. SISD (Single Instruction, Single Data)

- **Description:**

A single processor executes one instruction at a time on a single stream of data.

A single processor executes one instruction at a time on a single stream of data”, we are describing the SISD (Single Instruction Single Data) computer architecture, which is the simplest type.

Single processor → There is only one CPU (processing unit).

One instruction at a time → The CPU can only fetch, decode, and execute one instruction in a given clock cycle (not multiple in parallel).

Single stream of data → That one instruction is applied to one set of data only, not multiple pieces of data at once.

- **Example:**

A traditional uniprocessor computer or a standard desktop/laptop CPU executing a single, sequential program.

2. SIMD (Single Instruction, Multiple Data)

- **Description:**

A single control unit issues the same instruction to multiple processing units, which then perform the operation on different data items simultaneously.

- **Example:**

Graphics Processing Units (GPUs) and vector processors that perform the same operation (e.g., adding pixels) on large sets of data.

A single control unit issues the same instruction to multiple processing units, which then perform the operation on different data items simultaneously.

Single control unit (CU): There is one unit that decides what instruction (like ADD, SUB, MULTIPLY) should be executed.

Multiple processing units (PUs): Instead of just one CPU, there are many processors (like small sub-CPU) that can perform the instruction in parallel.

Same instruction: All processors execute the same instruction at the same time.

Different data items: Each processor works on its own piece of data

3. MISD (Multiple Instruction, Single Data)

- **Description:**

Multiple processing units execute different instructions on the same stream of data.

Multiple processing units (PUs): There is more than one processor working in the system.

Different instructions: Each processor performs a different operation (e.g., one does addition, another does subtraction, another does multiplication).

Same stream of data: All processors are working on the same input data.

- **Example:**

This architecture is not commonly seen in commercial systems, but systolic arrays are sometimes cited as an example for tasks like signal processing.

4. MIMD (Multiple Instruction, Multiple Data)

- **Description:**

Multiple processors operate independently, each executing its own set of instructions on its own separate data streams.

Multiple processors operate independently, each executing its own set of instructions on its own separate data streams.

Multiple processors: There are several CPUs (or cores).

Operate independently: Each processor doesn't have to wait for the others — they run on their own.

Different instructions: Each processor can execute a completely different instruction (e.g., one is adding while another is sorting).

Different data streams: Each processor works with its own data.

- **Example:**

Modern multiprocessor systems, including smartphones and laptops, are MIMD, allowing for true multitasking.

Type	Description	Example	Usage
SISD (Single Instruction Single Data)	One processor executes one instruction on one data stream.	Traditional single-core CPUs.	Used in most standard desktop and laptop computers.
SIMD (Single Instruction Multiple Data)	One instruction is executed simultaneously on multiple data streams.	Graphics Processing Units (GPUs).	Suitable for tasks like image processing and scientific computing.
MIMD (Multiple Instruction Multiple Data)	Multiple processors execute different instructions on different data streams.	Multicore processors (Intel i7, AMD Ryzen).	Used in parallel processing, supercomputing, and multicore systems.
MISD (Multiple Instruction Single Data)	Multiple processors execute different instructions on the same data stream.	Rarely used; found in specific fault-tolerant systems.	Used in redundant systems for error checking and critical applications.

Different Types of Languages

Different Programming Languages

```
01010010001111
0000000110101
01000110111101
```

Machine language



```
mov sec ebx
mov six e9c
mov rax rdx
```

Assembly language



```
#include <stdio.h>
int main( ) {
printf ("Hello World");
return 0; }
```

High-level language



Machine Language

Definition

- Machine language is the **lowest-level programming language**, consisting of binary code (0s and 1s).
- It is the **native language of the CPU**, directly executed without translation or interpretation.
- Each instruction in machine language corresponds to a specific operation of the processor.

Characteristics of Machine Language

- **Binary Format:** Instructions are represented as sequences of bits (0s and 1s).
- **Hardware-Specific:** Machine language instructions are specific to a particular processor or CPU family (e.g., Intel x86, ARM).
- **Difficult to Write and Understand:** Coding in machine language is complex, error-prone, and difficult to debug because it involves directly dealing with binary numbers.

Advantages and Disadvantages of Machine Language

Advantages	Disadvantages
Direct execution by the CPU.	Difficult and error-prone to write.
Fast and efficient performance.	Hardware-specific, not portable.
No need for a compiler or assembler.	Hard to debug and understand.

2. Assembly Language

Definition

- Assembly language is a **low-level programming language** that uses **mnemonics** (short symbolic names) instead of binary codes to represent machine instructions.
- It is a more **human-readable form** of machine language, acting as an intermediary between high- level languages (like C, Java) and machine language.
- Programs written in assembly language must be translated into machine language by a program called an **assembler**.

Characteristics of Assembly Language

- **Mnemonic Codes:** Uses short names for operations, such as `MOV`, `ADD`, `SUB`, `JMP`, which are easier to read and remember than binary codes.
 - **Symbolic Addressing:** Allows the use of labels and symbolic names for memory locations and registers.
 - **Hardware-Dependent:** Assembly language is specific to the CPU architecture (e.g., x86, ARM).
 - **Requires an Assembler:** Converts the assembly code into machine language for execution by the
- ## Advantages and Disadvantages of Assembly Language

Advantages	Disadvantages
Easier to read and understand than machine language.	Still complex compared to high-level languages.
Efficient and can produce fast-executing code.	Not portable; specific to a particular CPU architecture.
Allows direct control of hardware resources.	Requires deep knowledge of the computer architecture.
Useful for writing performance-critical code (e.g., device drivers, embedded systems).	More error-prone than high-level programming.

Machine Language vs Assembly Language

Aspect	Machine Language	Assembly Language
Representation	Binary code (0s and 1s)	Mnemonics (e.g., MOV, ADD, SUB)
Readability	Not human-readable	More human-readable
Translation	Directly executed by the CPU	Needs an assembler to translate to machine code
Ease of Use	Difficult and error-prone	Easier than machine language, but still complex
Portability	CPU-specific	CPU-specific
Example Instruction	10101000 00000101	MOV AX, 5

Use Cases of Machine Language and Assembly Language

- 1. Machine Language:
 - Used in situations where **direct hardware manipulation** is needed.
 - Found in **bootloaders**, firmware, and microcode.
- 2. Assembly Language:
 - Commonly used in **embedded systems**, device drivers, real-time systems, and system-level programming.
 - Utilized for writing **performance-critical code** where speed and efficiency are paramount.
 - Often employed in scenarios requiring **direct hardware control**, such as programming microcontrollers or writing BIOS code.

What is a Number?

A number is a mathematical value used for counting or measuring or labelling objects. Numbers are used to performing arithmetic calculations. Examples of numbers are natural numbers, whole numbers, rational and irrational numbers, etc. 0 is also a number that represents a null value.

A number has many other variations such as even and odd numbers, prime and composite numbers. Even and odd terms are used when a number is divisible by 2 or not, whereas prime and composite differentiate between the numbers that have only two factors and more than two factors, respectively.

In a number system, these numbers are used as digits. 0 and 1 are the most common digits in the number system, that are used to represent binary numbers. On the other hand, 0 to 9 digits are also used for other number systems. Let us learn here the types of number systems.

Types of Number Systems

There are various types of number systems in mathematics. The four most common number system types are:

1. Decimal number system (Base- 10)
2. Binary number system (Base- 2)
3. Octal number system (Base-8)
4. Hexadecimal number system (Base- 16)

Example

Write $(14)_{10}$ as a binary number.

Solution:

2	14	
2	7	0
2	3	1
	1	1

© BYJAL.COM

Base 2 Number System Example

$$\therefore (14)_{10} = 1110_2$$

Example: Convert 215_8 into decimal.

Solution:

$$215_8 = 2 \times 8^2 + 1 \times 8^1 + 5 \times 8^0$$

$$= 2 \times 64 + 1 \times 8 + 5 \times 1$$

$$= 128 + 8 + 5$$

$$= 141_{10}$$

i. Convert Decimal 156 to Binary and Hexadecimal

1. Decimal to Binary

- o $156 \div 2 = 78$, remainder **0**
- o $78 \div 2 = 39$, remainder **0**
- o $39 \div 2 = 19$, remainder **1**
- o $19 \div 2 = 9$, remainder **1**
- o $9 \div 2 = 4$, remainder **1**
- o $4 \div 2 = 2$, remainder **0**
- o $2 \div 2 = 1$, remainder **0**
- o $1 \div 2 = 0$, remainder **1**

Binary equivalent: 10011100_2

2. Decimal to Hexadecimal

- o $156 \div 16 = 9$, remainder **12** (C in hexadecimal)
- o $9 \div 16 = 0$, remainder **9**

Hexadecimal equivalent: $9C_{16}$

ii. Convert Binary 110101 to Decimal and Octal

1. Binary to Decimal

$$(1 \times 2^5) + (1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\ (1 \times 2^5) + (1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\ (32) + (16) + (0) + (4) + (0) + (1) = 53$$

Decimal equivalent: 53_{10}

2. Binary to Octal (Group into triplets from right: 110 101)

o $110_2 = 6_8$

o $101_2 = 5_8$

Octal equivalent: 65_8

Floating-Point Arithmetic and IEEE Standards

Floating-point arithmetic is essential in computer systems for representing **real numbers** and performing operations on them. Unlike integers, real numbers include fractions, decimals, and very large or small numbers, which require a special format for representation. The **IEEE 754 standard** is the most widely used specification for floating-point arithmetic in modern computer systems.

1. What is Floating-Point Arithmetic?

Definition

- Floating-point arithmetic involves the representation and manipulation of **real numbers** (numbers with fractional parts) using a specific format that allows a wide range of values.
- It uses a **scientific notation-like** representation where numbers are expressed in the form: $\text{Number} = \text{Sign} \times \text{Mantissa} \times 2^{\text{Exponent}}$

Components of Floating-Point Representation

1. **Sign Bit:**
 - o Represents whether the number is positive or negative.
 - o A sign bit of **0** indicates a positive number, and a sign bit of **1** indicates a negative number.
2. **Mantissa (Significant):**
 - o Represents the significant digits of the number.
 - o It determines the **precision** of the floating-point number.
3. **Exponent:**
 - o Indicates the power of **2** by which the mantissa is multiplied.
 - o The exponent allows the representation of both very large and very small numbers.

Example of Floating-Point Representation

Consider the number **-13.25** in decimal:

1. Convert to binary: **-1101.01**.
2. Normalize to scientific form: **-1.10101×2^3** .
3. Here, the **sign bit** is 1 (negative), the **mantissa** is **1.10101**, and the **exponent** is **3**.

2. IEEE 754 Standard for Floating-Point Arithmetic

The **IEEE 754 standard** defines the representation and behavior of floating-point numbers in computer systems. It ensures consistency and accuracy across different platforms and hardware.

Key Features of IEEE 754 Standard

- Provides formats for **single-precision (32-bit)** and **double-precision (64-bit)** floating-point numbers.
- Defines special values like **Infinity**, **NaN (Not a Number)**, **zero**, and **denormalized numbers**.
- Specifies rules for **rounding**, **overflow**, **underflow**, and **exception handling**.

1. Why Use Signed Number Representation?

- In binary, there are only **0s** and **1s**, making it straightforward to represent **unsigned integers** (only positive numbers).
- To represent **negative integers**, a special encoding scheme is needed because binary numbers inherently do not have a "negative" sign.
- Signed number representations solve this issue by assigning specific binary patterns to represent negative values.

2. Sign and Magnitude Representation (Basic Method)

Before diving into 1's and 2's complement, let's briefly look at **sign and magnitude**, a simpler but less efficient method:

- The **leftmost bit (Most Significant Bit or MSB)** is used as the **sign bit**:
 - 0** for positive numbers.
 - 1** for negative numbers.
- The remaining bits represent the **magnitude** (absolute value) of the number.

Comparison: 1's Complement vs. 2's Complement

Feature	1's Complement	2's Complement
Representation	Invert all bits (1 → 0, 0 → 1).	Invert all bits and add 1.
Zero Representation	Two zeros: +0 (0000 0000), -0 (1111 1111).	One zero: 0000 0000.
Arithmetic	More complex (requires handling two zeros).	Simpler (no need to handle two zeros).

Range (8-bit)	-127 to +127.	-128 to +127.
Usage	<i>Rarely used in modern systems.</i>	<i>Standard in modern computer systems.</i>

- *Overview of System Bus and Bus Interconnection*
-